Workshop 2: Cross-validation and model selection

TECH6

Richard Foltyn
NHH Norwegian School of Economics

October 22, 2026

See GitHub repository for notebooks and data:

https://github.com/richardfoltyn/TECH6-H26

Contents

1 Exercise 1: Selecting and evaluating a polynomial model 1

2 Exercise 2: Categorical features 7

3 Exercise 3: Predicting loan interest rates with PCR 11

Exercise 1: Selecting and evaluating a polynomial model

Consider the model

$$y_i = f(x_i) + \epsilon_i, \qquad f(x) = \sin(2\pi(x + 0.1)),$$

where x_i is uniform on $[0, 1]$ and the independently and identically distributed (i.i.d.) errors satisfy $\epsilon_i \overset{\text{iid}}{\sim} N(0, 0.5^2)$. For this exercise, assume that we don't know the true model and are instead approximating f with a polynomial of degree D, i.e.,

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_D x_i^D + u_i,$$

where u_i is the residual not explained by the polynomial.

In this exercise, we use cross-validation (CV) to select a polynomial degree, refit the selected model, and evaluate its predictions on independent observations. We first compute the validation scores for each degree, then reproduce the same selection using higher-level scikit-learn functions.

Setup

The true function, sample generator, and plotting functions are provided in the following cells. We use 100 training observations for all model selection and fitting. We generate an independent test sample only after selecting and fitting the models.

[1]:
```
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import default_rng
```

[2]:
```
def fcn(x):
    """
    Evaluate the true sine relationship.

    Parameters
    ----------
    x : array-like
        Input values

    Returns
    -------
    array
        True function values
    """
    return np.sin(2 * np.pi * (x + 0.1))
```

[3]:
```
def generate_sample(n_obs, sigma, rng=None):
    """
    Generate a sample with uniform inputs and normally distributed errors.

    Parameters
    ----------
    n_obs : int
        Number of observations
    sigma : float
        Standard deviation of the errors
    rng : generator, optional
        Random number generator used for the draws

    Returns
    -------
    X : array
        Feature matrix with one column
    y : array
        Observed outcomes
    """
    if rng is None:
        rng = default_rng(seed=1234)

    # Draw uniform inputs
    x = rng.uniform(size=n_obs)
    # Draw normally distributed errors
    epsilon = rng.normal(scale=sigma, size=n_obs)
    # Add the errors to the true function values
    y = fcn(x) + epsilon
    # Reshape x to a feature matrix with one column
    X = x[:, None]
    return X, y
```

[4]:
```
def plot_sample(X, y):
    """
    Plot the training observations and true function.

    Parameters
    ----------
    X : array
        Feature matrix with one column
    y : array
        Observed outcomes
    """
    x_grid = np.linspace(0, 1, 301)
    _fig, ax = plt.subplots(figsize=(4.5, 3), layout='constrained')
```

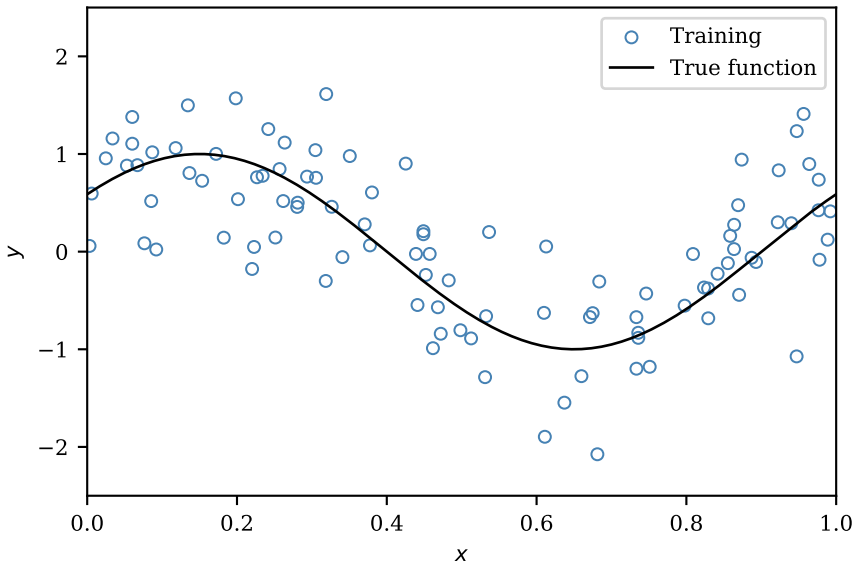
```
    ax.scatter(X[:, 0], y, s=20, c='none', ec='steelblue', lw=0.75, label='Training')
    ax.plot(x_grid, fcn(x_grid), c='black', lw=1, label='True function')
    ax.set(xlabel='$x$', ylabel='$y$', xlim=(0, 1), ylim=(-2.5, 2.5))
    ax.legend()
    return ax
```

[5]:
```
# Number of observations
n_obs = 100

# Standard deviation of the errors
sigma = 0.5

# Generate the sample
X, y = generate_sample(n_obs, sigma)

# Plot the sample
_ = plot_sample(X, y)
```



Part 1 — Constructing the pipeline

As a first step, build a `Pipeline` called `pipe` with these named steps:

1. `input_scale`: use `StandardScaler` to center and scale the input.
2. `poly`: use `PolynomialFeatures` with `degree=1` and `include_bias=False`.
3. `scale`: standardize the expanded features with another `StandardScaler`.
4. `lr`: fit `LinearRegression` with `fit_intercept=True`.

Explain why the scalers must be inside the pipeline when we use cross-validation.

Test that the pipeline works by fitting it to the sample generated above.

[6]:
```
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

# TODO: Build pipe with the specified steps

# TODO: Test pipeline by fitting it on X and y.
```

Part 2 — Computing MSEs for one degree

In this part, you are asked to implement a function that returns the average training and validation MSEs across CV folds for a given polynomial degree.

1. Define a CV splitter using `KFold` with ten shuffled folds and a fixed seed of 1234.
2. Complete the function `compute_average_mse()` below. Use `cross_validate()` to compute the training and validation mean squared errors (MSEs) for each fold. The function takes an estimator, the training data, and a CV splitter, and returns the mean training and validation MSEs across folds.

   Hint: Pass `scoring='neg_mean_squared_error'` and `return_train_score=True` as arguments to `cross_validate()`.

   Hint: Don't forget to negate the scores before returning the MSEs.

Test your function implementation with the pipeline from Part 1, the CV splitter from (1), and the training data generated earlier.

[8]:
```python
from sklearn.model_selection import cross_validate


def compute_average_mse(estimator, X, y, cv):
    """
    Compute mean training and validation MSEs across CV folds.

    Parameters
    ----------
    estimator : estimator
        Regression model or pipeline
    X : array-like
        Explanatory variables
    y : array-like
        Response variable
    cv : splitter
        Cross-validation splitter

    Returns
    -------
    mse_train : float
        Mean training MSE across folds
    mse_validate : float
        Mean validation MSE across folds
    """
    # TODO: Run cross_validate() with training scores and negative MSE scoring.

    # TODO: Negate and average the training and validation scores.

    # TODO: Return the two mean MSEs.


# TODO: Define a CV splitter with ten shuffled folds and a random state of 1234.

# TODO: Test compute_average_mse() with the pipeline, sample, and CV splitter.
```

Part 3 — Plotting the validation curve

We now want to use the function `compute_average_mse()` implemented in Part 2 to compute the mean training and validation MSEs for polynomial degrees 1 through 15. We are therefore "manually" computing the validation curve. Recall that the validation curve plots the training and validation scores as a function of a hyperparameter, in this case the polynomial degree.

1. Loop over degrees 1 through 15. For each degree, you can either recreate the pipeline with the new degree or use `pipe.set_params()` to change `poly__degree` on an existing pipeline (see code template below).

   Call your `compute_average_mse()` function with the same CV splitter defined in Part 2 to compute the mean training and validation MSEs for each degree and store these.

2. Find and report the degree with the smallest mean validation MSE.

3. Use the supplied `plot_validation_curve()` function to plot the training and validation MSEs as a function of the polynomial degree.

[12]:
```python
def plot_validation_curve(degrees, mse_train, mse_validation, best_degree):
    """
    Plot mean training and validation MSEs against polynomial degree.

    Parameters
    ----------
    degrees : array-like
        Candidate polynomial degrees
    mse_train : array-like
        Mean training MSE for each degree
    mse_validation : array-like
        Mean validation MSE for each degree
    best_degree : int
        Degree with the smallest mean validation MSE
    """
    _fig, ax = plt.subplots(figsize=(5.5, 3), layout='constrained')
    ax.plot(degrees, mse_train, c='black', marker='x', ms=4, label='Training')
    ax.plot(
        degrees, mse_validation, c='steelblue', marker='o', ms=4, label='Validation'
    )
    ax.axvline(best_degree, ls=':', lw=0.75, c='black')
    ax.set(xlabel='Polynomial degree', ylabel='Mean CV MSE', xticks=degrees)
    ax.legend()
```

[13]:
```python
# Degrees of the polynomial features to consider
degrees = np.arange(1, 16)

for degree in degrees:
    # Update the degree of the polynomial features in the pipeline
    pipe.set_params(poly__degree=degree)

    # TODO: Compute the mean training and validation MSEs for the current degree

    # TODO: Store the mean training and validation MSEs for each degree

# TODO: Find and report the degree with the smallest mean validation MSE

# TODO: Plot the validation curve using the supplied `plot_validation_curve()` function
```

Part 4 — Using `validation_curve()`

Instead of computing the validation curve manually, we can use the convenience function `validation_curve()`.

1. Repeat the degree comparison using `validation_curve()`. Use the same pipeline, candidate degrees, CV splitter, and negative-MSE scorer.

   What do the rows and columns of the returned arrays represent?

2. Average each returned score array across folds and negate the scores. Check that the resulting training and validation MSEs agree with Part 3 using `np.allclose()`.

[16]:
```python
from sklearn.model_selection import validation_curve

# TODO: Call validation_curve() with param_name='poly__degree'.

# TODO: Average across folds, negate the scores, and compare with Part 3.
```

Part 5 — Selecting and refitting automatically

Next, we want to select the polynomial degree and refit the model automatically using `GridSearchCV` instead of computing the validation curve manually.

1. Use `GridSearchCV` to select among the same degrees, using the same pipeline, folds, and scorer.

   Hint: Set `refit=True` to automatically refit the best model on the full training data.

2. Print the selected degree using the `best_params_` attribute and its mean validation MSE from the `best_score_` attribute of the `GridSearchCV` object. Remember to negate the score to obtain the MSE.

   Compare the printed values with the results from Part 3.

[20]:
```python
from sklearn.model_selection import GridSearchCV

# TODO: Create grid_search with the same degree grid, splitter, and scorer.

# TODO: Fit the search on X and y, then print the selected degree and CV MSE.
```

Part 6 — Comparing fitted models

You are now asked to visually compare the fitted models with degree 1, the CV-selected degree, and degree 15.

1. Fit the comparison models with degrees 1 and 15 on all training observations, and use `GridSearchCV`'s `best_estimator_` attribute for the selected best model.

   Hint: Instead of defining new pipelines, you can use `clone()` to create unfitted copies of the pipeline for degrees 1 and 15, and then adjust their `poly__degree` parameter using `set_params(poly__degree=...)`.

2. Use the supplied function below to plot a fitted model together with the sample and true function. Call it once for each model to create three separate plots.

3. Explain how the plots relate to the validation curve.

[23]:
```python
def plot_fitted_model(X, y, model, title):
    """
    Plot one fitted model with the sample and true function.

    Parameters
    ----------
    X : array
        Training feature matrix with one column
    y : array
        Training outcomes
    model : estimator
        Fitted regression model or pipeline
    title : str
        Plot title
    """
    ax = plot_sample(X, y)
    x_grid = np.linspace(0, 1, 301)
    y_hat = model.predict(x_grid[:, None])
```

```
    ax.plot(x_grid, y_hat, c='darkorange', lw=1.25, label='Fitted polynomial')
    ax.set_title(title)
    ax.legend()
```

[24]:
```
from sklearn.base import clone

# Fit degree-1 copy of pipe.
model_d1 = clone(pipe).set_params(poly__degree=1).fit(X, y)

# TODO: Fit degree-15 copy of pipe.

# TODO: Save best model from grid_search.best_estimator_ as best_model

# TODO: Call plot_fitted_model() for each of the three models.
```

Part 7 — Independent evaluation

We used cross-validation to choose the polynomial degree and then fitted the selected model on all 100 training observations. We now evaluate predictions using a new test sample. These observations were not used to choose the degree or fit any of the models.

Because these data are simulated, we can generate a new test sample instead of setting aside some of the original observations.

1. Use `generate_sample()` to generate 10,000 independent test observations with `default_rng(5678)`, the same true function, and the same noise level.
2. Use `mean_squared_error()` to report training and test MSE for the three models fitted in Part 6.

Explain:

1. Which model has the lowest training MSE? Which has the lowest test MSE?
2. Compare degree 15 with the CV-selected degree. Does the lower training MSE translate into better predictions on the test sample? Explain the difference.

[29]:
```
import pandas as pd
from sklearn.metrics import mean_squared_error

# TODO: Generate the test sample X_test, y_test

# TODO: Compute and report each model's training and test MSE.
```

Exercise 2: Categorical features

In this exercise, we practice fitting models that include categorical features in the regressor matrix $\mathbf{X}$ using the Ames housing dataset. There are several categorical variables in this data that come in two different kinds:

1. Nominal categorical variables capture qualitative categories. For numerical purposes, we often encode these as integer values, but any particular integer representation is arbitrary. It is a serious mistake to include such integer-valued categorical variables as *continuous* features in a model.

   In the Ames housing data, the `Neighborhood` column is one such nominal categorical variable (stored as a string containing the neighborhood name).

2. Ordinal categorical variables represent data that have a natural ordering, but the distance between any two values can be arbitrary.

   For example, in the Ames housing data the `OverallQuality` variable is an ordinal categorical variable on a scale of 1–10. While a value of 2 is clearly better than a value of 1, it is up to interpretation how much better.

   Such variables can be included as categorical *dummy variables* in a model, not imposing any particular interpretation on the distance between two values. Alternatively, such variables can be included as *continuous* variables, which imposes how the distance is interpreted. For example, including an ordinal categorical variable as a linear term in a model imposes that moving from 1 to 2 has the same effect on the outcome variable as moving from 2 to 3.

Part 1 — Creating the estimation sample

We load the Ames data the same way as we did in the lectures:

[32]:
```
import pandas as pd

# Use local data/ folder
DATA_PATH = '../../data'

filename = f'{DATA_PATH}/ames_houses.csv'
df = pd.read_csv(filename, sep=',')
```

First, perform the following data preprocessing steps:

1. Drop all observations that have any missing values for `SalePrice`, `OverallQuality`, or `Neighborhood`.
2. Tabulate how many neighborhoods have fewer than 40 observations, and drop observations from these neighborhoods from the dataset.

Part 2 — Train-test split

Split the data into a training and a test sample, assigning 40% of observations to the test sample, and report the number of observations in each set.

Stratify your split by `Neighborhood` to preserve approximately the same neighborhood proportions in both samples. Use the `stratify` argument of `train_test_split()` and specify a random seed of 1234.

You can pass the whole `DataFrame` to `train_test_split()` instead of individual `X` and `y` values. The function then returns two DataFrames, one containing the training data and one containing the test data.

Part 3 — Exploratory data analysis

Use only `df_train` for the following exploratory analysis so that test outcomes do not guide model development:

1. Compute the correlations of `SalePrice` with the other numerical variables. Which explanatory variable has the highest correlation?
2. Create a bar chart of `OverallQuality`, showing the number of observations for each quality level 1–10.

   Hint: Create a `Series` with the number of observations per quality level and plot it using `Series.plot.bar()`.

3. Create a box plot showing `SalePrice` by `OverallQuality`.

   Hint: You can use `DataFrame.plot.box()` for this.

4. Create a box plot showing `SalePrice` by `Neighborhood`.

Part 4 — Predicting price by neighborhood

Create an estimator pipeline to fit the model

$$SalePrice_{ij} = \mu_j + \epsilon_{ij}$$

where i indexes observations and j indexes neighborhoods. That is, the sale price for a house i in neighborhood j is predicted to be the average sale price μ_j in that neighborhood.

In order to estimate this model, you'll have to convert the categorical string variable `Neighborhood` to a set of 0/1 dummy variables. This can be achieved using the `OneHotEncoder` as a step in a pipeline.

Fit the model on the training data, and report the RMSE both on the training and the test sample.

This is a predetermined baseline. Use training data and CV scores to choose between the quality models in the remaining parts.

Part 5 — Polynomial in overall quality

Fit the model

$$SalePrice_i = \beta_0 + \beta_1 OverallQuality_i + \beta_2 OverallQuality_i^2 + \cdots + \beta_d OverallQuality_i^d + \epsilon_i$$

for polynomial degrees $d = 1, 2, \ldots, 9$. This treats quality as a numeric predictor and imposes a smooth relationship across its ordered levels. In the next part, we compare this with a separate mean for each quality category.

Use the template below and `cross_validate()` to compute mean training and validation RMSEs. Include `StandardScaler` both before and after `PolynomialFeatures(degree=d, include_bias=False)`, followed by `LinearRegression(fit_intercept=True)`. Keep the scalers inside the pipeline.

Use `scoring='neg_root_mean_squared_error'` and `return_train_score=True`. The arrays under `train_score` and `test_score` contain negative RMSEs for each fold; `test_score` refers to the CV validation folds.

Plot the mean training and validation RMSEs and report the degree with the smallest mean validation RMSE. Reuse `cv_quality` for the dummy-model comparison. The training observations are already in random order after the train-test split, so we keep that order when assigning the five folds.

[47]:
```
import numpy as np
from sklearn.model_selection import KFold

features = ['OverallQuality']
target = 'SalePrice'
X_train, y_train = df_train[features], df_train[target]
X_test, y_test = df_test[features], df_test[target]

degrees = np.arange(1, 10)
cv_quality = KFold(n_splits=5)
rmse_train_mean = []
rmse_validate_mean = []

for d in degrees:
    # TODO: Create the scaling + polynomial features + regression pipeline.

    # TODO: Compute training and validation scores using cross_validate().
```

```
    # scores = cross_validate(
    #     estimator=...,
    #     X=X_train, y=y_train,
    #     scoring=...,
    #     cv=cv_quality,
    #     return_train_score=True,
    #     n_jobs=4
    # )

    # TODO: Negate and average the scores across folds.
    # rmse_train_mean.append(np.mean(-scores['train_score']))
    # rmse_validate_mean.append(np.mean(-scores['test_score']))

    pass
```

Part 6 — Quality dummies

Compare the polynomial models with a separate mean for each quality category:

$$SalePrice_{ij} = \mu_j + \epsilon_{ij},$$

where j indexes the quality levels 1–10. A house with quality j is predicted to have the average training sale price of houses with that quality.

Create a pipeline with `OneHotEncoder` and `LinearRegression`, then evaluate it using the same observations, `cv_quality` splitter, and RMSE scorer as in Part 5. The dummy model has no polynomial degree to tune, but CV lets us compare its predictive performance with the polynomial models.

Recreate the validation curve and add horizontal lines for the dummy model's mean training and validation RMSEs. Compare its scores with those of degree 9. Why might these agree when all ten quality levels occur in every training fold?

Choose between the best polynomial model and the dummy model using their mean validation RMSEs, before inspecting their test predictions.

[51]:
```
# TODO: Create pipeline with dummy variables + estimation

# TODO: Run cross-validation
# scores = cross_validate(
#     estimator=...,
#     X=X_train, y=y_train,
#     scoring=...,
#     cv=cv_quality,
#     return_train_score=True,
#     n_jobs=4
# )

# TODO: Compute average RMSE over all CV splits
# rmse_train_dummies_mean = np.mean(-scores['train_score'])
# rmse_validate_dummies_mean = np.mean(-scores['test_score'])
```

Part 7 — Comparing test predictions

Refit the selected polynomial and the dummy model on the whole training sample and compute their test RMSEs. Report the results for both predetermined models.

Does the test ranking agree with the CV ranking from Part 6? How large is the difference in test RMSE? Discuss what this comparison tells us without revising the model choice using the test results.

Exercise 3: Predicting loan interest rates with PCR

In this exercise, we build a model to predict a loan's interest rate from loan and borrower characteristics. We use principal component regression (PCR) to combine related predictors, and cross-validation to decide whether polynomial features and dimension reduction improve predictions.

We use data on 10,000 funded loans from OpenIntro's Lending Club dataset, stored in the file `lending_club.csv`. The loans were issued in the first quarter of 2018.

Our target is `interest_rate`, the annual interest rate in percent. We use the following eleven predictors. Counts and loan term enter as numeric variables. Homeownership and income verification enter as indicators.

Predictor	Description
`loan_amount`	Loan principal in US dollars
`term`	Loan term, either 36 or 60 months
`annual_income`	Applicant's annual income in US dollars
`debt_to_income`	Monthly debt payments as a percentage of monthly income
`homeowner`	1 for mortgage or outright ownership; 0 for renting
`income_verified`	1 if the source or amount of income was verified; 0 otherwise
`credit_history_years`	Issue year minus earliest credit-line year
`inquiries_last_12m`	Number of credit inquiries in the preceding 12 months
`open_credit_lines`	Number of open credit lines
`total_credit_limit`	Existing credit limit excluding mortgages, in US dollars
`credit_utilization`	Existing credit balance / limit, excluding mortgages

Credit utilization is a fraction: 0.5 means 50%. Both the balance and the limit exclude mortgages. These variables describe existing credit accounts across lenders, including accounts outside Lending Club. We interpret them as the applicant's credit position at application, before the loan in that row is issued.

Setup

We load the data and define the predictor list below. The setup keeps individual applications so that income and debt-to-income describe one applicant's finances. It removes observations missing the target or a selected predictor. The resulting DataFrame, `df_data`, contains 8,503 loans.

All model development will use the training sample; the test sample is reserved for Part 5.

[55]:
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Path to CSV in data/ folder
DATA_PATH = '../../data/lending-club'
df_raw = pd.read_csv(f'{DATA_PATH}/lending_club.csv')

# Select features and target variable for regression
features = [
    'loan_amount',
    'term',
    'annual_income',
    'debt_to_income',
    'homeowner',
    'income_verified',
    'credit_history_years',
    'inquiries_last_12m',
    'open_credit_lines',
    'total_credit_limit',
    'credit_utilization',
```

```
]
target = 'interest_rate'

# Keep individual applications (drop joint applications)
df_individual = df_raw.query("application_type == 'individual'")
# Keep only observations with non-missing values for the selected features and target
df_data = df_individual.dropna(subset=[*features, target])

print(f'Loans in the teaching sample: {len(df_raw):,}')
print(f'Usable observations: {len(df_data):,}')
```

```
Loans in the teaching sample: 10,000
Usable observations: 8,503
```

Part 1 — Splitting and inspecting the sample

We first set aside a test sample to evaluate predictions on unseen loans. We then inspect the training predictors' scales and correlations to see why standardization and PCA may be useful.

1. Use the prepared `df_data` sample and `train_test_split()` to reserve 30% for testing, with `random_state=1234`. Name the resulting objects `X_train`, `X_test`, `y_train`, and `y_test`. Report the number of training and test observations.
2. Using only the training sample, report each predictor's mean, standard deviation, minimum, and maximum.

   Why would PCA on the unscaled predictors be sensitive to their units?
3. Create the correlation heatmap by calling `plot_correlations(X_train)` using the provided function below. It computes the correlation matrix and reads the predictor names from `X_train` automatically.

[56]:
```
def plot_correlations(X):
    """
    Plot an annotated heatmap of predictor correlations.

    Parameters
    ----------
    X : dataframe
        Training predictors with named columns
    """
    df_corr = X.corr()

    fig, ax = plt.subplots(figsize=(6, 6), layout='constrained')
    heatmap = ax.imshow(df_corr, cmap='RdBu_r', vmin=-1, vmax=1)

    # Label the rows and columns with the predictor names
    positions = range(len(df_corr.columns))
    ax.set_xticks(positions, labels=df_corr.columns, rotation=45, ha='right')
    ax.set_yticks(positions, labels=df_corr.columns)

    # Display each correlation inside its cell
    for i in positions:
        for j in positions:
            correlation = df_corr.iloc[i, j]
            text_color = 'white' if abs(correlation) > 0.5 else 'black'
            ax.text(
                j,
                i,
                f'{correlation:.2f}',
                ha='center',
                va='center',
```

```
                c=text_color,
                fontsize=8,
            )

    fig.colorbar(heatmap, ax=ax, label='Correlation', shrink=0.5)
    ax.set_title('Training predictor correlations')
    plt.show()
```

[57]:
```
from sklearn.model_selection import train_test_split

# TODO: Split df_data into training and test sets and report their sizes.

# TODO: Inspect training predictor descriptive statistics

# TODO: Create correlation heatmap using plot_correlations(X_train).
```

Part 2 — Building the PCR pipeline

We next combine preprocessing, PCA, and regression in a single pipeline.

Build a `Pipeline` called `pipe_pcr` with these named steps:

1. `input_scale`: `StandardScaler()`.
2. `poly`: `PolynomialFeatures()` with `degree=2` and `include_bias=False`.
3. `scale`: another `StandardScaler()`.
4. `pca`: `PCA` with `n_components=0.9` and `svd_solver='full'`.
5. `lr`: `LinearRegression()` with `fit_intercept=True`.

Fit the pipeline on the training sample. Report the number of expanded features, the number of retained components, and their total explained predictor variance.

Hint: Use `n_features_in_`, `n_components_`, and `explained_variance_ratio_` from the fitted PCA step.

[61]:
```
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler

# TODO: Build and fit pipe_pcr.

# TODO: Inspect the fitted PCA step and report its dimension and explained variance.
```

Part 3 — Selecting degree and retention

We now use cross-validation to choose the polynomial degree and the amount of predictor variance retained by PCA.

Use `GridSearchCV` to select the optimal polynomial degree and PCA variance threshold jointly.

- Create a parameter grid with the following values:
  - Candidate degrees: `[1, 2, 3]`.
  - Candidate variance thresholds: `[0.7, 0.8, 0.9, 0.95, 0.99, 0.999, 0.9999]`.
- Use R^2 as the scoring metric, `scoring='r2'`.
- Define a CV splitter using `KFold` with 5 shuffled folds and a fixed seed of 1234.

- Create the `GridSearchCV` object with the argument `refit=True` so that the best model is refitted on the full training sample after the search.

Save the best model as `best_pcr` and call the provided `tabulate_cv_results(pcr_cv)` function to present the CV R^2 scores in a table with degrees as rows and variance thresholds as columns.

Use the table to discuss whether greater polynomial flexibility or more retained variance always improves prediction.

[64]:
```python
def tabulate_cv_results(pcr_cv):
    """
    Tabulate the CV results from a GridSearchCV object for PCR with polynomial features.

    Parameters
    ----------
    pcr_cv : GridSearchCV
        Fitted GridSearchCV object

    Returns
    -------
    results : DataFrame
        Mean CV R² by polynomial degree and PCA variance threshold
    """
    keys = ['param_poly__degree', 'param_pca__n_components', 'mean_test_score']
    results = pd.DataFrame(pcr_cv.cv_results_)[keys]
    results = results.rename(
        columns={
            'mean_test_score': 'CV R²',
            'param_poly__degree': 'Poly degree',
            'param_pca__n_components': 'PCA variance',
        }
    )

    results = results.pivot(
        index='Poly degree', columns='PCA variance', values='CV R²'
    ).round(3)

    return results
```

[65]:
```python
from sklearn.model_selection import GridSearchCV, KFold

# Candidate polynomial degrees and PCA variance thresholds
degrees = [1, 2, 3]
thresholds = [0.7, 0.8, 0.9, 0.95, 0.99, 0.999, 0.9999]
# Scoring metric for GridSearchCV
scoring = 'r2'

# TODO: Define cv_splitter, the parameter grid, and pcr_cv.

# TODO: Fit the search and report the selected parameters and mean CV R².

# TODO: Save best_estimator_ as best_pcr.
```

Part 4 — Comparing prediction methods

We now compare PCR with polynomial regression without dimension reduction. Does PCR improve predictions relative to using the polynomial features directly?

1. Build a new `Pipeline` called `pipe_poly` with these named steps:
    1. `input_scale`: `StandardScaler()`,

2. `poly`: `PolynomialFeatures()` with `degree=2` and `include_bias=False`,
3. `scale`: another `StandardScaler()`, and
4. `lr`: `LinearRegression()` with `fit_intercept=True`.

2. Run a new `GridSearchCV` search called `poly_cv` over degrees 1, 2, and 3, using the same training observations, `cv_splitter`, and `scoring`.

   Save the refitted best model as `best_poly`.

3. Use the provided `compare_cv_results(pcr_cv, poly_cv)` function to compare the highest PCR CV R^2 across variance thresholds with the CV R^2 without PCA for each degree. Does discarding components help more at some degrees than at others?

Based on the validation results, which method would you choose for predicting interest rates?

[70]:
```python
def compare_cv_results(pcr_cv, poly_cv):
    """
    Compare validation scores with and without PCA at each polynomial degree.

    Parameters
    ----------
    pcr_cv : search
        Fitted PCR grid search
    poly_cv : search
        Fitted polynomial regression grid search

    Returns
    -------
    dataframe
        Best PCR and polynomial CV scores at each degree
    """
    df_pcr = pd.DataFrame(pcr_cv.cv_results_)
    pcr_scores = df_pcr.groupby('param_poly__degree')['mean_test_score'].max()

    df_poly = pd.DataFrame(poly_cv.cv_results_)
    poly_scores = df_poly.set_index('param_poly__degree')['mean_test_score']

    df_compare = pd.DataFrame({'PCR': pcr_scores, 'Without PCA': poly_scores})
    df_compare.index.name = 'Degree'
    return df_compare.round(3)
```

[71]:
```python
# TODO: Build pipe_poly

# TODO: Run GridSearchCV over degrees 1, 2, and 3.

# TODO: Save the best estimator as best_poly

# TODO: Call compare_cv_results(pcr_cv, poly_cv) and interpret the table.
```

Part 5 — Evaluating unseen loans

We finish by evaluating both fitted models on unseen loans and plotting PCR's predictions against observed rates.

1. Use `best_pcr` and `best_poly` to predict interest rates for the test set. Save the PCR predictions as `y_test_hat` and the polynomial predictions as `y_test_poly_hat`. Report the test R^2 for both models using `r2_score()`.
2. Use `scatter()` to plot observed test rates on the horizontal axis and PCR predictions on the vertical axis. Add a dashed line where predicted and observed rates are equal using `axline()`.
3. Briefly describe where the model overpredicts or underpredicts interest rates.

[74]:
```
from sklearn.metrics import r2_score

# TODO: Predict with best_pcr and save the predictions as y_test_hat.

# TODO: Predict with best_poly and save the predictions as y_test_poly_hat.

# TODO: Compute and report the test R² for both models.

# TODO: Plot PCR predictions against observed test rates, with an equality line.
```